

SCOREBOARD ET TOMASULO

M2 Secil : KINN9HU
Pascal SAINRAT



UNIVERSITÉ
TOULOUSE III
PAUL SABATIER

FACULTÉ SCIENCES
ET INGÉNIERIE

Ordonnancement dynamique

Introduction

- Le compilateur essaie de minimiser les dépendances présentes dans le programme.
- Le compilateur essaie d'éviter que les dépendances restantes produisent des suspensions par un ordonnancement statique des instructions mais cet ordonnancement n'est valide que pour un pipeline donné.
- L'ordonnancement dynamique (matériel) réordonne l'exécution des instructions pour éviter les suspensions.
- Avantages:
 - permet de traiter des cas où les dépendances ne sont connues que dynamiquement (accès mémoire).
 - permet d'exécuter efficacement un code produit pour un autre pipeline.
- Inconvénient:
 - coût matériel

L'ordonnancement dynamique

principe

- Si une instruction j dépend d'une instruction i , ses opérandes ne peuvent être lus => suspension du pipeline
- Mais si l'instruction $j+1$ n'a pas de dépendances, elle pourrait être exécutée (pipeline avec plusieurs UFs)
- Problème des exceptions qui sont imprécises puisque les instructions se terminent dans le désordre. On va garder cette hypothèse pour l'instant.
- L'étage de décodage est divisé en 2 : lancement (décodage, test aléas structurels) et lecture des opérandes (attente des données, lecture)
- Lancement toujours dans l'ordre. Ensuite suspension ou passage à l'étage suivant en doublant une autre.

Mécanismes matériels

ordonnancement dynamique

- Pourquoi matériellement à l'exécution?
 - Fonctionne quand les dépendances ne sont pas connues à la compilation
 - Compilateur plus simple
 - Code plus indépendant de l'architecture

- Idée clé: Permettre à des instructions bloquées de s'exécuter

DIVD F0, F2, F4

ADDD F10, F0, F8

SUBD F8, F8, F14

- Permettre l'exécution non ordonnée
 - Terminaison dans le désordre
 - Exceptions imprécises
- Exécution non ordonnée quand il y a suffisamment de ressources et aucune dépendance de données.

Mécanismes matériels:

ordonnancement dynamique

- L'exécution non ordonnée divise l'étage de décodage en 2 :
 1. Lancement—décode les instructions, détecte les aléas structurels
 2. Lecture opérandes—attendre qu'il n'y ait plus d'aléas de données, puis lire les opérandes

Implications

DIVD **F0**, F2, F4
ADDD F10, **F0**, **F8**
SUBD **F8**, F8, F14

- Terminaison non ordonnée : aléa EAL
 - Instruction bloquée jusqu'à ce que l'autre ait lu son opérande
- Détection aléas EAE
 - Blocage jusqu'à ce que la première se termine
- Plusieurs unités d'exécution
- Le tableau de marques garde la trace des dépendances des états et des opérations
- Le tableau de marques remplace DE, EX, ER par 4 étages

Les quatre étages (1)

1. Lancement—décode les instructions & test des aléas structurels (DI1)

Si une UF est libre pour cette instruction et qu'il n'y a pas d'instructions actives ayant le même registre destination (EAE), le tableau lance l'instruction vers l'UF et met à jour les infos. Si aléa EAE, blocage lancement y compris pour les instructions suivantes.

Lecture des opérandes—attendre qu'il n'y ait plus d'aléas de données et lire les opérandes (DI2)

Un opérande source est disponible s'il n'y a pas d'instruction en cours qui va y écrire. Quand les opérandes sources sont disponibles, le tableau dit à l'unité de lire les registres. Résolution dynamique des aléas LAE à cette étape, les instructions peuvent être envoyées à l'exécution dans le désordre.

Les quatre étages (1)

1. Lancement—décode les instructions & test des aléas structurels (DI1)

Si une UF est libre pour cette instruction et qu'il n'y a pas d'instructions actives ayant le même registre destination (EAE), le tableau lance l'instruction vers l'UF et met à jour les infos. Si aléa EAE, blocage lancement y compris pour les instructions suivantes.

2. Lecture des opérandes—attendre qu'il n'y ait plus d'aléas de données et lire les opérandes (DI2)

Un opérande source est disponible s'il n'y a pas d'instruction en cours qui va y écrire. Quand les opérandes sources sont disponibles, le tableau dit à l'unité de lire les registres. Résolution dynamique des aléas LAE à cette étape, les instructions peuvent être envoyées à l'exécution dans le désordre.

Les quatre étages (2)

3. Exécution—(EX)

L'unité fonctionnelle commence l'exécution lorsqu'elle a reçu les opérandes. Quand le résultat est prêt, elle notifie au tableau de marques qu'elle a fini son exécution.

4. Ecriture du résultat—fin de l'exécution (ER)

Une fois que le tableau sait que l'UF a terminé son exécution, le tableau teste les aléas EAL. S'il n'y en a pas, il écrit le résultat sinon il bloque l'instruction (et l'UF).

Exemple:

```
DIVD    F0,F2,F4
ADDD    F10,F0,F8
SUBD    F8,F8,F14
```

Le tableau bloque SUBD jusqu'à ce que ADDD ait lu ses opérandes

Les quatre étages (2)

3. Exécution—(EX)

L'unité fonctionnelle commence l'exécution lorsqu'elle a reçu les opérandes. Quand le résultat est prêt, elle notifie au tableau de marques qu'elle a fini son exécution.

4. Ecriture du résultat—fin de l'exécution (ER)

Une fois que le tableau sait que l'UF a terminé son exécution, le tableau teste les aléas EAL. S'il n'y en a pas, il écrit le résultat sinon il bloque l'instruction (et l'UF).

Exemple :

```
DIVD    F0,F2,F4
ADDD    F10,F0,F8
SUBD    F8,F8,F14
```



Le tableau bloque SUBD jusqu'à ce que ADDD ait lu ses opérandes

Les trois parties du tableau

1. Etat des instructions—Un des 4 précédents

2. Etat des UF—9 champs pour chaque UF

Occupé—Indique si l'UF est occupée ou non

Op—Opération à exécuter dans (e.g., + or −)

Rd—Registre destination

Rs1, Rs2—Registres sources

Ufs1, Ufs2—UF produisant les registres sources Rs1, Rs2

Ps1, Ps2—Drapeaux indiquant si Rs1 et Rs2 n'ont pas été lus

3. Etat des registres résultats—indique quelle UF va écrire chaque registre. Vide s'il n'y en a pas.

Tableau de marques: Tests et actions

UF: unité fonctionnelle utilisée par l'instruction; D: registre destination de l'instruction
S1, S2: registres source de l'instruction; op: opération correspondant à l'instruction;

Etat instruction	Attendre jusqu'à	Actions à effectuer
Lancement	Occupé(UF) == Non && Résultat('D') vide	$\text{Occupé(UF)} \leftarrow \text{oui}; \text{Op(UF)} \leftarrow \text{op};$ $\text{Rd(UF)} \leftarrow \text{'D'}; \text{Rs1} \leftarrow \text{'S1'}; \text{Rs2} \leftarrow \text{'S2'};$ $\text{Ufs1(UF)} \leftarrow \text{Résultat('S1')};$ $\text{Ufs2(UF)} \leftarrow \text{Résultat('S2')};$ $\text{Ps1} \leftarrow \text{Non Ufs1}; \text{Ps2} \leftarrow \text{Non Ufs2};$ $\text{Résultat('D')} \leftarrow \text{FU};$
Lecture opérandes	(Ps1==Oui) && (Ps2==Oui)	$\text{Ps1} \leftarrow \text{Non}; \text{Ps2} \leftarrow \text{Non}$
Exécution terminée	Fin de l'exécution dans l'UF	
Ecriture résultat	$\forall f,$ $(\text{Rs1}(f) \neq \text{Rd(UF)} \text{ ou } \text{Ps1}(f) == \text{Non})$ et $(\text{Rs2}(f) \neq \text{Rd(UF)} \text{ ou } \text{Ps2}(f) == \text{Non})$	$\forall f, (\text{si } \text{Ufs1}(f) = \text{UF} \text{ alors } \text{Ps1} \leftarrow \text{Oui});$ $\forall f, (\text{si } \text{Ufs2}(f) = \text{UF} \text{ alors } \text{Ps2} \leftarrow \text{Oui});$ $\text{Résultat(Rs1(UF))} \leftarrow 0;$ $\text{Occupé(UF)} \leftarrow \text{Non};$

Tableau de marques (Remarques)

- La performance dépend de la taille du tableau
 - Problèmes de temps de cycle et de coût
- Le nombre d'UFs influe sur les aléas structurels
 - Problème du nombre de bus et de ports sur le tableau
 - Partager les bus pour les UF non pipelinée (DIV)
- EAL et EAE suspendent les instructions
- L'exemple a été pris sur un seul bloc de base
 - Si arrêt sur branchement, peu d'instructions indépendantes
 - Si prédiction de branchement, risque d'EAE et EAL supplémentaires (plusieurs itérations de boucles)

L'ALGORITHME DE TOMASULO



UNIVERSITÉ
TOULOUSE III
PAUL SABATIER

FACULTÉ SCIENCES
ET INGÉNIERIE

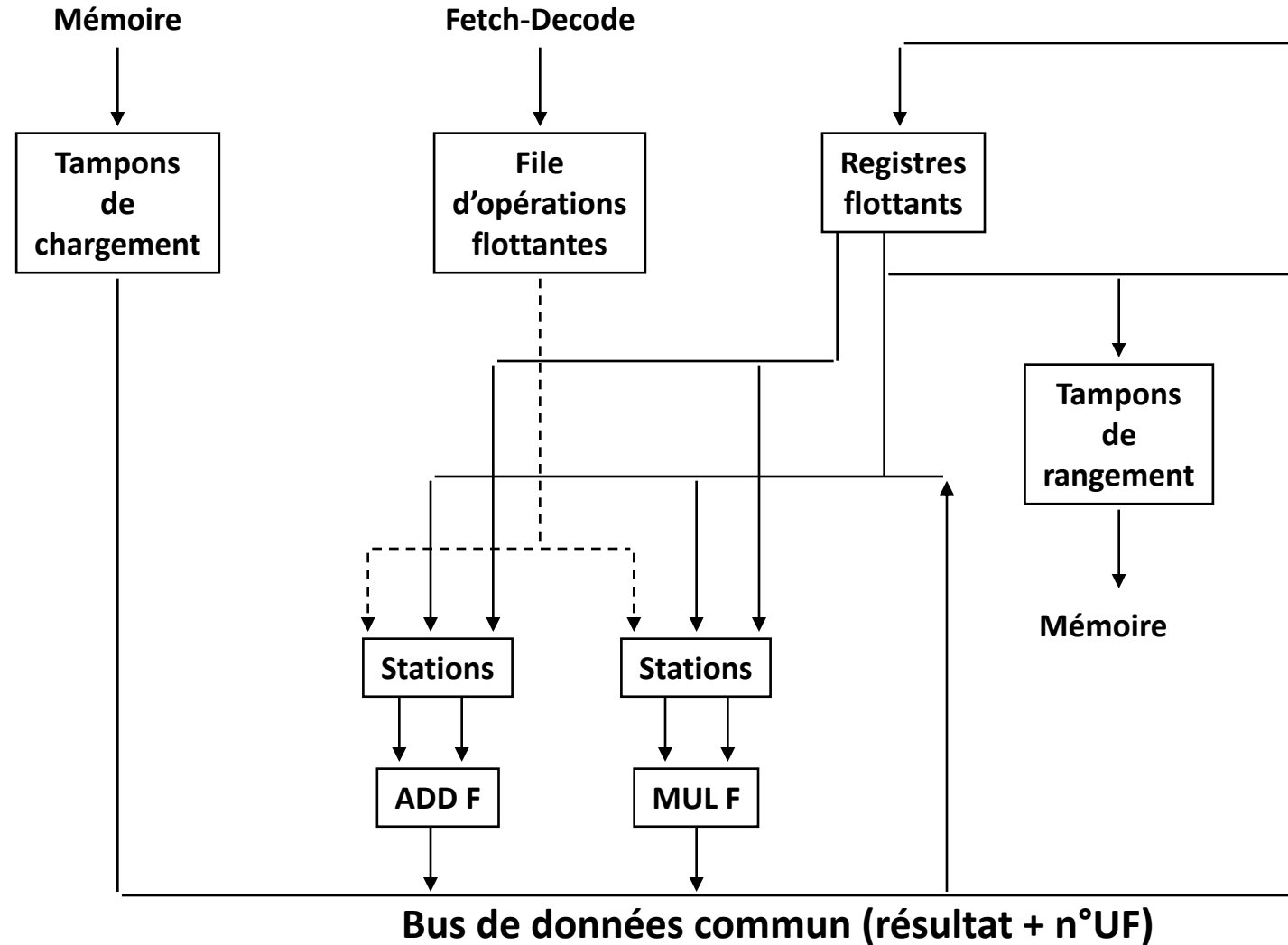
Résumé sur le tableau de marques

- Amélioration de 1,7 sur le compilateur mais pas de cache
- Limitations du tableau de marques du 6600
 - Pas de bypass
 - Limité aux instructions d'un bloc de base (petite fenêtre)
 - Nombre d'unités fonctionnelles (hasards structurels)
 - Attente des aléas EAL
 - Préviend les aléas EAE

Un autre algorithme dynamique : l'algorithme de Tomasulo

- Pour l'IBM 360/91 environ 3 ans après le CDC 6600
- But : Haute performance avec compilateurs spéciaux
- Différences entre les jeux d'instructions de l'IBM 360 & du CDC 6600
 - IBM n'a que 2 registres/instr contre 3 dans le CDC 6600
 - IBM a 4 registres FP contre 8 dans le CDC 6600
- Differences entre l'algo de Tomasulo & le tableau de marques
 - Le contrôle & les tampons sont distribués auprès des UFs; appelés “stations de réservation”
 - Les numéros de registres dans les instructions sont remplacés par des pointeurs vers les stations de réservation
 - Les registres sont renommés matériellement pour éviter les EAL et les EAE
 - Des bus de données communs diffusent les résultats à toutes les UFs
 - Les Load et les Store sont traités de la même façon que les UFs

Tomasulo – Structures matérielles



Constitution d'une station de réservation

Op	—Opération à effectuer dans l'UF (par exemple, +)
Q_{s1}, Q_{s2}	—Stations de réservation produisant les registres source
V_{s1}, V_{s2}	—Valeurs des opérandes sources
R_{s1}, R_{s2}	—Drapeaux indiquant que V_{s1}, V_{s2} sont prêts
Occupé	—indique que la station et l'UF sont occupées

Etat du registre résultat—Indique, pour chaque registre, quelle UF va produire le résultat s'il y en a une.

Les 3 étages de l'algorithme de Tomasulo

1. **Lancement**—Lire l'instruction depuis la file d'opérations flottante
Si une station de réservation est vide, envoi des opérandes à la station s'ils sont disponibles.
2. **Exécution**—opération sur les opérandes (EX)
Quand les 2 opérandes sont prêts, exécution
S'ils ne sont pas prêts, attendre le résultat sur le bus
3. **Ecriture du résultat**—fin de l'exécution (WB)
Ecriture sur le bus de données commun vers toutes les stations
Libérer la station de réservation.

Tomasulo

Les étapes de l'algorithme

Etat de l'instruction	Attente jusqu'à	Actions à effectuer
Lancement	Station ou tampon disponible	Si (Reg[S1].Qi != 0) {SR[r].Qs1 $\leftarrow$ Reg[S1].Qi} Sinon {SR[r].Vs1 $\leftarrow$ Reg[S1]; SR[r].Qs1 $\leftarrow$ □□□□} Si (Reg[S2].Qi != 0) {SR[r].Qs2 $\leftarrow$ Reg[S2].Qi} Sinon {SR[r].Vs2 $\leftarrow$ Reg[S2]; SR[r].Qs2 $\leftarrow$ □□□□} SR[r].occupé $\leftarrow$ oui; Registre[D].Qi $\leftarrow$ r ;
Exécution	(SR[r].Qs1=0) et SR[r].Qs2=0)	Aucune. Les opérandes sont dans Vs1 et Vs2.
Ecriture résultat	Exécution terminée de r et bus résultat disponible.	$\forall x$, (si Reg[x].Qi = r) {Fx $\leftarrow$ résultat ; Reg[x].Qi $\leftarrow$ □□} □ $\forall x$, (si SR[x].Qs1 = r) {SR[x].Vs1 $\leftarrow$ résultat, SR[x].Qs1 $\leftarrow$ □} $\forall x$, (si SR[x].Qs2 = r) {SR[x].Vs2 $\leftarrow$ résultat, SR[x].Qs2 $\leftarrow$ □} $\forall x$, (si Range[x].Qs1 = r) {Range[x].V $\leftarrow$ résultat, Range[x].Qs1 $\leftarrow$ □} SR[r].occupé $\leftarrow$ non ;

Tomasulo — Exemple de boucle

```

Loop: LD      F0      0      R1
      MULTDF4 F0      F2
      SD      F4      0      R1
      SUBI    R1      R1     #8
      BNEZ    R1      Loop
  
```

- La multiplication dure 4 cycles
- Le premier load provoque un échec dans le cache (8 cycles)

Tomasulo — Exemple de boucle

<u>Etat instruction</u>					<i>Exécution</i>		<i>Ecriture</i>	<u>Stations de réservation L/S</u>			
<i>Instruction</i>		<i>S1</i>	<i>S2</i>	<i>itération</i>	<i>Lancée</i>	<i>finie</i>	<i>Résultat</i>	<i>Nom</i>	<i>Occupé</i>	<i>Adresse</i>	
LD	F0	0	R1	1				Load1			
MULTD	F4	F0	F2	1				Load2			
SD	F4	0	R1	1				Load3			<i>Qi</i>
LD	F0	0	R1	2				Store1			
MULTD	F4	F0	F2	2				Store2			
SD	F4	0	R1	2				Store3			
<u>Stations de réservation</u>											
	<i>Temps</i>	<i>Nom</i>	<i>Occupé</i>	<i>Op</i>	<i>V_{S1}</i>	<i>V_{S2}</i>	<i>Q_{S1}</i>	<i>Q_{S2}</i>			
		Add1									
		Add2									
		Add3									
		Mult1									
		Mult2									
<u>Etat registre résultat</u>											
<i>Cycle</i>		<i>R1</i>		<i>F0</i>	<i>F2</i>	<i>F4</i>	<i>F6</i>	<i>F8</i>	<i>F10</i>	<i>F12</i>	<i>... F30</i>
			<i>Qi</i>								

Hypothèses: l'addition dure 2 cycles, la multiplication 10 cycles et la division 40 cycles

Exemple de boucle Cycle 1

<u>Etat instruction</u>						Exécution	Ecriture	<u>Stations de réservation L/S</u>			
Instruction	S1	S2	itération	Lancée	finie	Résultat	Nom	Occupé	Adresse		
LD F0	0 R1		1	1			Load1	Oui	80		
MUL F4	F0 F2		1				Load2	Non			
SD F4	0 R1		1				Load3	Non		Qi	
LD F0	0 R1		2				Store1	Non			
MUL F4	F0 F2		2				Store2	Non			
SD F4	0 R1		2				Store3	Non			
<u>Stations de réservation</u>											
	Temps	Nom	Occupé	Op	V _{S1}	V _{S2}	Q _{S1}	Q _{S2}	Code:		
	0	Add1	Non						LD	F0	0 R1
	0	Add2	Non						MULTD	F4	F0 F2
	0	Add3	Non						SD	F4	0 R1
	0	Mult1	Non						SUBI	R1	R1 #8
	0	Mult2	Non						BNEZ	R1	Loop
<u>Etat registre résultat</u>											
Cycle	R1		F0	F2	F4	F6	F8	F10	F12	...	F30
1	80	Qi	Load1								

Exemple de boucle Cycle 2

Etat instruction					Exécution	Ecriture	Stations de réservation L/S					
Instruction		S1	S2	itération	Lancée	finie	Résultat	Nom	Occupé	Adresse		
LD	F0	0	R1	1	1			Load1	Oui	80		
MULTD	F4	F0	F2	1	2			Load2	Non			
SD	F4	0	R1	1				Load3	Non		Qi	
LD	F0	0	R1	2				Store1	Non			
MULTD	F4	F0	F2	2				Store2	Non			
SD	F4	0	R1	2				Store3	Non			
Stations de réservation												
	Temps	Nom	Occupé	Op	VS1	VS2	QS1	QS2	Code:			
	0	Add1	Non						LD	F0	0	R1
	0	Add2	Non						MULTD	F4	F0	F2
	0	Add3	Non						SD	F4	0	R1
	0	Mult1	Oui	MULTD		R(F2)	Load1		SUBI	R1	R1	#8
	0	Mult2	Non						BNEZ	R1	Loop	
Etat registre résultat												
Cycle		R 1		F0	F2	F4	F6	F8	F10	F12	...	F30
2		80	Qi	Load1		Mult1						

Exemple de boucle Cycle 3

Etat instruction						Exécution	Ecriture					
Instruction		S1	S2	iteration	Lanct	terminée	Résultat		Occupé	Adresse		
LD	F0	0	R1	1	1			Load1	Oui	80		
MULTD	F4	F0	F2	1	2			Load2	Non			
SD	F4	0	R1	1	3			Load3	Non		Qi	
LD	F0	0	R1	2				Store1	Oui	80	Mult1	
MULTD	F4	F0	F2	2				Store2	Non			
SD	F4	0	R1	2				Store3	Non			
Stations de réservation												
	Temps	Nom	Occupé	Op	V _{S1}	V _{S2}	Q _{S1}	Q _{S2}	Code:			
	0	Add1	Non						LD	F0	0	R1
	0	Add2	Non						MULTD	F4	F0	F2
	0	Add3	Non						SD	F4	0	R1
	0	Mult1	Oui	MULTD		R(F2)	Load1		SUBI	R1	R1	#8
	0	Mult2	Non						BNEZ	R1	Loop	
Etat registre résultat												
Cycle		R1		F0	F2	F4	F6	F8	F10	F12	...	F30
3		80	Qi	Load1		Mult1						

Exemple de boucle Cycle 4

Etat instruction						Exécution	Ecriture					
Instruction		S1	S2	iteration	Lanct	terminée	Résultat		Occupé	Adresse		
LD	F0	0	R1	1	1			Load1	Oui	80		
MULTD	F4	F0	F2	1	2			Load2	Non			
SD	F4	0	R1	1	3			Load3	Non		Qi	
LD	F0	0	R1	2				Store1	Oui	80	Mult1	
MULTD	F4	F0	F2	2				Store2	Non			
SD	F4	0	R1	2				Store3	Non			
Stations de réservation												
	Temps	Nom	Occupé	Op	V _{S1}	V _{S2}	Q _{S1}	Q _{S2}	Code:			
	0	Add1	Non						LD	F0	0	R1
	0	Add2	Non						MULTD	F4	F0	F2
	0	Add3	Non						SD	F4	0	R1
	0	Mult1	Oui	MULTD		R(F2)	Load1		SUBI	R1	R1	#8
	0	Mult2	Non						BNEZ	R1	Loop	
Etat registre résultat												
Cycle		R1		F0	F2	F4	F6	F8	F10	F12	...	F30
4		72	Qi	Load1		Mult1						

Exemple de boucle Cycle 5

Etat instruction						Exécution	Ecriture					
Instruction		S1	S2	iteration	Lanct	terminée	Résultat		Occupé	Adresse		
LD	F0	0	R1	1	1			Load1	Oui	80		
MULTD	F4	F0	F2	1	2			Load2	Non			
SD	F4	0	R1	1	3			Load3	Non		Qi	
LD	F0	0	R1	2				Store1	Oui	80	Mult1	
MULTD	F4	F0	F2	2				Store2	Non			
SD	F4	0	R1	2				Store3	Non			
Stations de réservation												
	Temps	Nom	Occupé	Op	V _{S1}	V _{S2}	Q _{S1}	Q _{S2}	Code:			
	0	Add1	Non						LD	F0	0	R1
	0	Add2	Non						MULTD	F4	F0	F2
	0	Add3	Non						SD	F4	0	R1
	0	Mult1	Oui	MULTD		R(F2)	Load1		SUBI	R1	R1	#8
	0	Mult2	Non						BNEZ	R1	Loop	
Etat registre résultat												
Cycle		R 1		F0	F2	F4	F6	F8	F10	F12	...	F30
5		72	Qi	Load1		Mult1						

Exemple de boucle Cycle 6

Etat instruction						Exécution	Ecriture					
Instruction		S1	S2	iteration	Lanct	terminée	Résultat	Occupé	Adresse			
LD	F0	0	R1	1	1		Load1	Oui	80			
MULTD	F4	F0	F2	1	2		Load2	Oui	72			
SD	F4	0	R1	1	3		Load3	Non			Qi	
LD	F0	0	R1	2	6		Store1	Oui	80		Mult1	
MULTD	F4	F0	F2	2			Store2	Non				
SD	F4	0	R1	2			Store3	Non				
Stations de réservation												
	Temps	Nom	Occupé	Op	V _{S1}	V _{S2}	Q _{S1}	Q _{S2}	Code:			
	0	Add1	Non						LD	F0	0	R1
	0	Add2	Non						MULTD	F4	F0	F2
	0	Add3	Non						SD	F4	0	R1
	0	Mult1	Oui	MULTD		R(F2)	Load1		SUBI	R1	R1	#8
	0	Mult2	Non						BNEZ	R1	Loop	
Etat registre résultat												
Cycle		R1		F0	F2	F4	F6	F8	F10	F12	...	F30
6		72	Qi	Load2		Mult1						

Exemple de boucle Cycle 7

Etat instruction						Exécution	Ecriture					
Instruction		S1	S2	iteration	Lanct	terminée	Résultat		Occupé	Adresse		
LD	F0	0	R1	1	1			Load1	Oui	80		
MULTD	F4	F0	F2	1	2			Load2	Oui	72		
SD	F4	0	R1	1	3			Load3	Non		Qi	
LD	F0	0	R1	2	6			Store1	Oui	80	Mult1	
MULTD	F4	F0	F2	2	7			Store2	Non			
SD	F4	0	R1	2				Store3	Non			
Stations de réservation												
	Temps	Nom	Occupé	Op	V _{S1}	V _{S2}	Q _{S1}	Q _{S2}	Code:			
	0	Add1	Non						LD	F0	0	R1
	0	Add2	Non						MULTD	F4	F0	F2
	0	Add3	Non						SD	F4	0	R1
	0	Mult1	Oui	MULTD		R(F2)	Load1		SUBI	R1	R1	#8
	0	Mult2	Oui	MULTD		R(F2)	Load2		BNEZ	R1	Loop	
Etat registre résultat												
Cycle		R1		F0	F2	F4	F6	F8	F10	F12	...	F30
7		72	Qi	Load2		Mult2						

Exemple de boucle Cycle 8

Etat instruction						Exécution	Ecriture					
Instruction		S1	S2	iteration	Lancée	terminée	Résultat		Occupé	Adresse		
LD	F0	0	R1	1	1		Load1		Oui	80		
MULTD	F4	F0	F2	1	2		Load2		Oui	72		
SD	F4	0	R1	1	3		Load3		Non		Qi	
LD	F0	0	R1	2	6		Store1		Oui	80	Mult1	
MULTD	F4	F0	F2	2	7		Store2		Oui	72	Mult2	
SD	F4	0	R1	2	8		Store3		Non			
Stations de réservation												
	Temps	Nom	Occupé	Op	V _{S1}	V _{S2}	Q _{S1}	Q _{S2}	Code:			
	0	Add1	Non						LD	F0	0	R1
	0	Add2	Non						MULTD	F4	F0	F2
	0	Add3	Non						SD	F4	0	R1
	0	Mult1	Oui	MULTD		R(F2)	Load1		SUBI	R1	R1	#8
	0	Mult2	Oui	MULTD		R(F2)	Load2		BNEZ	R1	Loop	
Etat registre résultat												
Cycle		R1		F0	F2	F4	F6	F8	F10	F12	...	F30
8		72	Qi	Load2		Mult2						

Exemple de boucle Cycle 9

Etat instruction						Exécution	Ecriture					
Instruction		S1	S2	iteration	Lancée	terminée	Résultat	Occupé	Adresse			
LD	F0	0	R1	1	1	9	Load1	Oui	80			
MULTD	F4	F0	F2	1	2		Load2	Oui	72			
SD	F4	0	R1	1	3		Load3	Non			Qi	
LD	F0	0	R1	2	6		Store1	Oui	80		Mult1	
MULTD	F4	F0	F2	2	7		Store2	Oui	72		Mult2	
SD	F4	0	R1	2	8		Store3	Non				
Stations de réservation												
	Temps	Nom	Occupé	Op	V _{S1}	V _{S2}	Q _{S1}	Q _{S2}	Code:			
	0	Add1	Non						LD	F0	0	R1
	0	Add2	Non						MULTD	F4	F0	F2
	0	Add3	Non						SD	F4	0	R1
	0	Mult1	Oui	MULTD		R(F2)	Load1		SUBI	R1	R1	#8
	0	Mult2	Oui	MULTD		R(F2)	Load2		BNEZ	R1	Loop	
Etat registre résultat												
Cycle		R 1		F0	F2	F4	F6	F8	F10	F12	...	F30
9		64	Qi	Load2		Mult2						

Exemple de boucle Cycle 10

Etat instruction					Exécution	Ecriture					
Instruction		<i>j</i>	<i>k</i>	<i>iteration</i>	<i>Lancée</i>	<i>terminée</i>	<i>Résultat</i>		Occupé	Adresse	
LD	F0	0	R1	1	1	9	10	Load1	Non		
MULTD	F4	F0	F2	1	2			Load2	Oui	72	
SD	F4	0	R1	1	3			Load3	Non		Qi
LD	F0	0	R1	2	6	10		Store1	Oui	80	Mult1
MULTD	F4	F0	F2	2	7			Store2	Oui	72	Mult2
SD	F4	0	R1	2	8			Store3	Non		
Stations de réservation											
	<i>Temps</i>	<i>Nom</i>	<i>Occupé</i>	<i>Op</i>	<i>V_{S1}</i>	<i>V_{S2}</i>	<i>Q_{S1}</i>	<i>Q_{S2}</i>	<i>Code:</i>		
	0	Add1	Non						LD	F0	0 R1
	0	Add2	Non						MULTD	F4	F0 F2
	0	Add3	Non						SD	F4	0 R1
	4	Mult1	Oui	MULTD	M(80)	R(F2)			SUBI	R1	R1 #8
	0	Mult2	Oui	MULTD		R(F2)	Load2		BNEZ	R1	Loop
Etat registre résultat											
<i>Cycle</i>		<i>R 1</i>		<i>F0</i>	<i>F2</i>	<i>F4</i>	<i>F6</i>	<i>F8</i>	<i>F10</i>	<i>F12</i>	... <i>F30</i>
10		64	Qi	Load2		Mult2					

Exemple de boucle Cycle 11

Etat instruction						Exécution	Ecriture					
Instruction		S1	S2	iteration	Lancée	terminée	Résultat		Occupé	Adresse		
LD	F0	0	R1	1	1	9	10	Load1	Non			
MULTD	F4	F0	F2	1	2			Load2	Non			
SD	F4	0	R1	1	3			Load3	Oui	64	Qi	
LD	F0	0	R1	2	6	10	11	Store1	Oui	80	Mult1	
MULTD	F4	F0	F2	2	7			Store2	Oui	72	Mult2	
SD	F4	0	R1	2	8			Store3	Non			
Stations de réservation												
	Temps	Nom	Occupé	Op	V _{S1}	V _{S2}	Q _{S1}	Q _{S2}	Code:			
	0	Add1	Non						LD	F0	0	R1
	0	Add2	Non						MULTD	F4	F0	F2
	0	Add3	Non						SD	F4	0	R1
	3	Mult1	Oui	MULTD	M(80)	R(F2)			SUBI	R1	R1	#8
	4	Mult2	Oui	MULTD	M(72)	R(F2)			BNEZ	R1	Loop	
Etat registre résultat												
Cycle		R1		F0	F2	F4	F6	F8	F10	F12	...	F30
11		64	Qi			Mult2						

Exemple de boucle Cycle 12

Etat instruction						Exécution	Ecriture					
Instruction		S1	S2	iteration	Lancée	terminée	Résultat		Occupé	Adresse		
LD	F0	0	R1	1	1	9	10	Load1	Non			
MULTD	F4	F0	F2	1	2			Load2	Non			
SD	F4	0	R1	1	3			Load3	Oui	64	Qi	
LD	F0	0	R1	2	6	10	11	Store1	Oui	80	Mult1	
MULTD	F4	F0	F2	2	7			Store2	Oui	72	Mult2	
SD	F4	0	R1	2	8			Store3	Non			
Stations de réservation												
	Temps	Nom	Occupé	Op	V _{S1}	V _{S2}	Q _{S1}	Q _{S2}	Code:			
	0	Add1	Non						LD	F0	0	R1
	0	Add2	Non						MULTD	F4	F0	F2
	0	Add3	Non						SD	F4	0	R1
	2	Mult1	Oui	MULTD	M(80)	R(F2)			SUBI	R1	R1	#8
	3	Mult2	Oui	MULTD	M(72)	R(F2)			BNEZ	R1	Loop	
Etat registre résultat												
Cycle		R1		F0	F2	F4	F6	F8	F10	F12	...	F30
12		64	Qi			Mult2						

Exemple de boucle Cycle 13

Etat instruction					Exécution		Ecriture					
Instruction		S1	S2	iteration	Lancée	terminée	Résultat	Occupé	Adresse			
LD	F0	0	R1	1	1	9	10	Load1	Non			
MULTD	F4	F0	F2	1	2			Load2	Non			
SD	F4	0	R1	1	3			Load3	Oui	64	Qi	
LD	F0	0	R1	2	6	10	11	Store1	Oui	80	Mult1	
MULTD	F4	F0	F2	2	7			Store2	Oui	72	Mult2	
SD	F4	0	R1	2	8			Store3	Non			
Stations de réservation												
	Temps	Nom	Occupé	Op	V _{S1}	V _{S2}	Q _{S1}	Q _{S2}	Code:			
	0	Add1	Non						LD	F0	0	R1
	0	Add2	Non						MULTD	F4	F0	F2
	0	Add3	Non						SD	F4	0	R1
	1	Mult1	Oui	MULTD	M(80)	R(F2)			SUBI	R1	R1	#8
	2	Mult2	Oui	MULTD	M(72)	R(F2)			BNEZ	R1	Loop	
Etat registre résultat												
Cycle		R1		F0	F2	F4	F6	F8	F10	F12	...	F30
13		64	Qi			Mult2						

Exemple de boucle Cycle 14

Etat instruction						Exécution	Ecriture					
Instruction		S1	S2	iteration	Lancée	terminée	Résultat		Occupé	Adresse		
LD	F0	0	R1	1	1	9	10	Load1	Non			
MULTD	F4	F0	F2	1	2	14		Load2	Non			
SD	F4	0	R1	1	3			Load3	Oui	64	Qi	
LD	F0	0	R1	2	6	10	11	Store1	Oui	80	Mult1	
MULTD	F4	F0	F2	2	7			Store2	Oui	72	Mult2	
SD	F4	0	R1	2	8			Store3	Non			
Stations de réservation												
	Temps	Nom	Occupé	Op	V _{S1}	V _{S2}	Q _{S1}	Q _{S2}	Code:			
	0	Add1	Non						LD	F0	0	R1
	0	Add2	Non						MULTD	F4	F0	F2
	0	Add3	Non						SD	F4	0	R1
	0	Mult1	Oui	MULTD	M(80)	R(F2)			SUBI	R1	R1	#8
	1	Mult2	Oui	MULTD	M(72)	R(F2)			BNEZ	R1	Loop	
Etat registre résultat												
Cycle		R 1		F0	F2	F4	F6	F8	F10	F12	...	F30
14		64	Qi			Mult2						

Exemple de boucle Cycle 15

Etat instruction					Exécution		Ecriture					
Instruction		S1	S2	iteration	Lancée	terminée	Résultat		Occupé	Adresse		
LD	F0	0	R1	1	1	9	10	Load1	Non			
MULTD	F4	F0	F2	1	2	14	15	Load2	Non			
SD	F4	0	R1	1	3			Load3	Oui	64	Qi	
LD	F0	0	R1	2	6	10	11	Store1	Oui	80	M(80)*R(F2)	
MULTD	F4	F0	F2	2	7	15		Store2	Oui	72	Mult2	
SD	F4	0	R1	2	8			Store3	Non			
Stations de réservation												
	Temps	Nom	Occupé	Op	V _{S1}	V _{S2}	Q _{S1}	Q _{S2}	Code:			
	0	Add1	Non						LD	F0	0	R1
	0	Add2	Non						MULTD	F4	F0	F2
	0	Add3	Non						SD	F4	0	R1
	0	Mult1	Non						SUBI	R1	R1	#8
	0	Mult2	Oui	MULTD	M(72)	R(F2)			BNEZ	R1	Loop	
Etat registre résultat												
Cycle		R1		F0	F2	F4	F6	F8	F10	F12	...	F30
15		64	Qi			Mult2						

Exemple de boucle Cycle 16

Etat instruction					Exécution	Ecriture						
Instruction		S1	S2	iteration	Lancée	terminée	Résultat		Occupé	Adresse		
LD	F0	0	R1	1	1	9	10	Load1	Non			
MULTD	F4	F0	F2	1	2	14	15	Load2	Non			
SD	F4	0	R1	1	3			Load3	Oui	64	Q1	
LD	F0	0	R1	2	6	10	11	Store1	Oui	80	M(80)*R(F2)	
MULTD	F4	F0	F2	2	7	15	16	Store2	Oui	72	M(72)*R(F2)	
SD	F4	0	R1	2	8			Store3	Non			
Stations de réservation												
	Temps	Nom	Occupé	Op	V _{S1}	V _{S2}	Q _{S1}	Q _{S2}	Code:			
	0	Add1	Non						LD	F0	0	R1
	0	Add2	Non						MULT	F4	F0	F2
	0	Add3	Non						SD	F4	0	R1
	0	Mult1	Oui	MULTD		R(F2)	Load3		SUBI	R1	R1	#8
	0	Mult2	Non						BNEZ	R1	Loop	
Etat registre résultat												
Cycle		R1		F0	F2	F4	F6	F8	F10	F12	...	F30
16		64	Qi			Mult1						

Exemple de boucle Cycle 17

Etat instruction						Exécution	Ecriture					
Instruction		S1	S2	iteration	Lancée	terminée	Résultat		Occupé	Adresse		
LD	F0	0	R1	1	1	9	10	Load1	Non			
MULTD	F4	F0	F2	1	2	14	15	Load2	Non			
SD	F4	0	R1	1	3			Load3	Oui	64	Qi	
LD	F0	0	R1	2	6	10	11	Store1	Oui	80	M(80)*R(F2)	
MULTD	F4	F0	F2	2	7	15	16	Store2	Oui	72	M(72)*R(F2)	
SD	F4	0	R1	2	8			Store3	Oui	64	Mult1	
Stations de réservation												
	Temps	Nom	Occupé	Op	V _{S1}	V _{S2}	Q _{S1}	Q _{S2}	Code:			
	0	Add1	Non						LD	F0	0	R1
	0	Add2	Non						MULT	F4	F0	F2
	0	Add3	Non						SD	F4	0	R1
	0	Mult1	Oui	MULTD		R(F2)	Load3		SUBI	R1	R1	#8
	0	Mult2	Non						BNEZ	R1	Loop	
Etat registre résultat												
Cycle		R1		F0	F2	F4	F6	F8	F10	F12	...	F30
17		64	Qi			Mult1						

Exemple de boucle Cycle 18

Etat instruction						Exécution	Ecriture					
Instruction		S1	S2	iteration	Lancée	terminée	Résultat		Occupé	Adresse		
LD	F0	0	R1	1	1	9	10	Load1	Non			
MULTD	F4	F0	F2	1	2	14	15	Load2	Non			
SD	F4	0	R1	1	3	18		Load3	Oui	64	Qi	
LD	F0	0	R1	2	6	10	11	Store1	Oui	80	M(80)*R(F2)	
MULTD	F4	F0	F2	2	7	15	16	Store2	Oui	72	M(72)*R(F2)	
SD	F4	0	R1	2	8			Store3	Oui	64	Mult1	
Stations de réservation												
	Temps	Nom	Occupé	Op	V _{S1}	V _{S2}	Q _{S1}	Q _{S2}	Code:			
	0	Add1	Non						LD	F0	0	R1
	0	Add2	Non						MULTD	F4	F0	F2
	0	Add3	Non						SD	F4	0	R1
	0	Mult1	Oui	MULTD		R(F2)	Load3		SUBI	R1	R1	#8
	0	Mult2	Non						BNEZ	R1	Loop	
Etat registre résultat												
Cycle		R 1		F0	F2	F4	F6	F8	F10	F12	...	F30
18		56	Qi			Mult1						

Exemple de boucle Cycle 19

Etat instruction					Exécution	Ecriture						
Instruction		S1	S2	iteration	Lancée	terminée	Résultat		Occupé	Adresse		
LD	F0	0	R1	1	1	9	10	Load1	Non			
MULTD	F4	F0	F2	1	2	14	15	Load2	Non			
SD	F4	0	R1	1	3	18	19	Load3	Oui	64	Qi	
LD	F0	0	R1	2	6	10	11	Store1	Non			
MULTD	F4	F0	F2	2	7	15	16	Store2	Oui	72	M(72)*R(F4)	
SD	F4	0	R1	2	8			Store3	Oui	64	Mult1	
Stations de réservation												
	Temps	Nom	Occupé	Op	V _{S1}	V _{S2}	Q _{S1}	Q _{S2}	Code:			
	0	Add1	Non						LD	F0	0	R1
	0	Add2	Non						MULTD	F4	F0	F2
	0	Add3	Non						SD	F4	0	R1
	0	Mult1	Oui	MULTD		R(F2)	Load3		SUBI	R1	R1	#8
	0	Mult2	Non						BNEZ	R1	Loop	
Etat registre résultat												
Cycle		R1		F0	F2	F4	F6	F8	F10	F12	...	F30
19		56	Qi			Mult1						

Exemple de boucle Cycle 20

Etat instruction						Exécution	Ecriture					
Instruction		S1	S2	iteration	Lancée	terminée	Résultat		Occupé	Adresse		
LD	F0	0	R1	1	1	9	10	Load1	Non			
MULTD	F4	F0	F2	1	2	14	15	Load2	Non			
SD	F4	0	R1	1	3	18	19	Load3	Oui	64	Qi	
LD	F0	0	R1	2	6	10	11	Store1	Non			
MULTD	F4	F0	F2	2	7	15	16	Store2	Oui	72	M(72)*R(F2)	
SD	F4	0	R1	2	8	20		Store3	Oui	64	Mult1	
Stations de réservation												
	Temps	Nom	Occupé	Op	V _{S1}	V _{S2}	Q _{S1}	Q _{S2}	Code:			
	0	Add1	Non						LD	F0	0	R1
	0	Add2	Non						MULTD	F4	F0	F2
	0	Add3	Non						SD	F4	0	R1
	0	Mult1	Oui	MULTD		R(F2)	Load3		SUBI	R1	R1	#8
	0	Mult2	Non						BNEZ	R1	Loop	
Etat registre résultat												
Cycle		R1		F0	F2	F4	F6	F8	F10	F12	...	F30
20		56	Qi			Mult1						

Exemple de boucle Cycle 21

Etat instruction					Exécution		Ecriture					
Instruction		S1	S2	iteration	Lancée	terminée	Résultat	Occupé	Adresse			
LD	F0	0	R1	1	1	9	10	Load1	Non			
MULTD	F4	F0	F2	1	2	14	15	Load2	Non			
SD	F4	0	R1	1	3	18	19	Load3	Oui	64	Qi	
LD	F0	0	R1	2	6	10	11	Store1	Non			
MULTD	F4	F0	F2	2	7	15	16	Store2	Non			
SD	F4	0	R1	2	8	20	21	Store3	Oui	64	Mult1	
Stations de réservation												
	Temps	Nom	Occupé	Op	V _{S1}	V _{S2}	Q _{S1}	Q _{S2}	Code:			
	0	Add1	Non						LD	F0	0	R1
	0	Add2	Non						MULTD	F4	F0	F2
	0	Add3	Non						SD	F4	0	R1
	0	Mult1	Oui	MULTD		R(F2)	Load3		SUBI	R1	R1	#8
	0	Mult2	Non						BNEZ	R1	Loop	
Etat registre résultat												
Cycle		R 1		F0	F2	F4	F6	F8	F10	F12	...	F30
21		56	Qi			Mult1						

Conclusion Tomasulo

- Empêche les aléas de ressources de registre
- Evite donc les aléas EAL et EAE du tableau de marques
- Bypass
- Réalise un déroulage de boucle dans le matériel
- Non limité aux blocs de base si prédiction de branchement mais problème avec les écritures dans le désordre des registres
- Reste aussi le problème des exceptions précises